COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	7/23/2026	**DATE SUBMITTED:**	7/26/2026
NAME:	Dimaguiba, Lorenzo James M.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

SEATWORK NO. 3.1 SIMPLY LINKED LIST

1. PROCEDURE OUTPUT

ILO A:
Simple implementation:

Screenshot

```
template <typename T>
class SimplyList{
    public:
        T data;
        SimplyList<T> *next = nullptr;
};
```

VSCODES > CPE_Dimaguiba > HOA3p1 > main-1.cpp > main()

```
#include <iostream>
#include "simply_LL.h"

int main(){

    //Creating an empty list
    SimplyList<char>* head = new SimplyList<char>;;
    SimplyList<char>* second = new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;
    SimplyList<char>* fifth = new SimplyList<char>;
    SimplyList<char>* sixth = new SimplyList<char>;


    //assign data:
    //node 1
    head->data = 'C';
    head->next = second;

    //node2
    second->data = 'P';
    second->next = third;

    //node3
    third->data = 'E';
    third->next = fourth;

    //node4
    fourth->data = '0';
    fourth->next = fifth;

     //node5
    fifth->data = '1';
    fifth->next = sixth;

    //node6
    sixth->data = '0';
    sixth->next = nullptr;

     return 0;
}
```

	Output: PS C:\Users\shosh\OneDrive\Documents\VSCODES> osoft-MIEngine-Pid-2ectqdwv.onn' '--dbgExe=C: PS C:\Users\shosh\OneDrive\Documents\VSCODES> PS C:\Users\shosh\OneDrive\Documents\VSCODES> PS C:\Users\shosh\OneDrive\Documents\VSCODES> osoft-MIEngine-Pid-1fnabf5g.vt5' '--dbgExe=C: PS C:\Users\shosh\OneDrive\Documents\VSCODES>
Discussion	**Right now, nothing shows up. This is because there hasn't been any functions relating to actually printing the string of characters put into the linked list. There should be a way to print them in order that they were set as well.**

Table 3-1. Output of Initial/Simple Implementation

Operation	Screenshot

Traversal	`template <typename T >`
	`void listTraversal(SimplyList<T>* head){`
	
	`    if (head == nullptr){`
	`        std::cout<<"List is empty"<<std::endl;`
	`        return;`
	`    }`
	
	`    while(head != nullptr){`
	`        std::cout<<head->data;`
	`        head = head->next;`
	`    }`
	
	`}`

Insertion at head	`template<typename T>`
	`void sllinserteHead(T newData, SimplyList<T>** currenthead){`
	`    //Allocate memory for the new node`
	`    SimplyList<T>* newNode = new SimplyList<T>;`
	
	`    //Put our data into the new node`
	`    newNode->data = newData;`
	
	`    //Set Next of the new node to point to the previous Head`
	`    newNode->next = *currenthead;`
	
	`    //Reset Head to point to the new node`
	`    *currenthead = newNode;`
	
	`}`

Insertion at any part of the list

```
template <typename T>
void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
   // Check if it is the head node (previous node is null)
    if (prevNode == nullptr){
        //If null, print "Previous node cannot be null."
        std::cout<<"Previous node cannot be null"<<std::endl;
        return;
    }
    //Allocate a new node
    SimplyList<T>* newNode = new SimplyList<T>;

    //Store data in the new node
    newNode->data = newData;

    //Point new node to the node previous node was pointing to
    newNode->next = prevNode->next;

    //Point previous node to the new node
    prevNode->next = newNode;

}
```

Insertion at the end

```
template <typename T>
void sllinsertEnd(T newData, SimplyList<T>**head){
   //1. Allocate new node
    SimplyList<T>* newNode = new SimplyList<T>;
    //Dereference to the head node
    SimplyList<T>* currentNode = *head;
    //3. Store data in new node
    newNode->data = newData;
    //Point next of new node to NULL
    newNode->next = nullptr;
    //Traverse the list until next of the node is null
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;

    }

    //Point the next of the current node to the new node
    currentNode->next = newNode;
}
```

Deletion of a node	

```
template<typename T>
void sllDeleteNode(T findData, SimplyList<T>** head){

    //check if there's nothing
    if (*head == nullptr){
        return;
    }

    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    //searching
    while(currNode!= nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    //data was not found:

    if(currNode == nullptr){
        return;
    }

    //if node to be deleted is the head?
    if(prevNode == nullptr){
        *head = currNode->next;
    }
    else{
        prevNode->next = currNode-> next;
    }

    //how to delete node?
    delete currNode;
}
```

Table 3-2. Code for the List Operations

a.	Source Code	`std::cout<<"original list:"<<std::endl;` `listTraversal(head);`
	Console	`original list:` `CPE101`

		Description: Traverses through the entire list. Outputs the elements or letters in order as they were assigned.

b.	Source Code	`//node insertion at head`
		`std::cout<<"\ninserting a new node at the head: \n";`
		`sllinserteHead('G',&head);`
		`listTraversal(head);`

	Console	`inserting a new node at the head:`
		`GCPE101`
		Description: Finds the head, connects the new node to the list and points to the next as the new head.

c.	Source Code	`std::cout<<"\ninserting a new node at the third: "<<std::endl;`
		`sllGeneralInsert('E', head->next->next);`
		`listTraversal(head);`

	Console	`inserting a new node at the third:`
		`GCPEE101`
		Description: It finds the third character and inserts 'E' next to it.

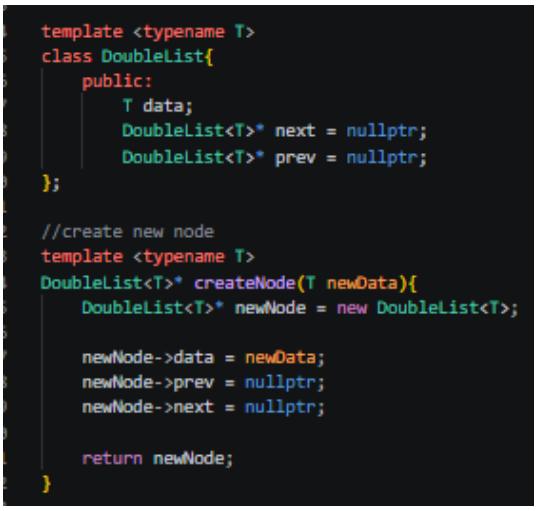
d.	Source Code	`std::cout<<"\ndeleting a node: \n"<<std::endl;`
		`sllDeleteNode('C', &head);`
		`listTraversal(head);`

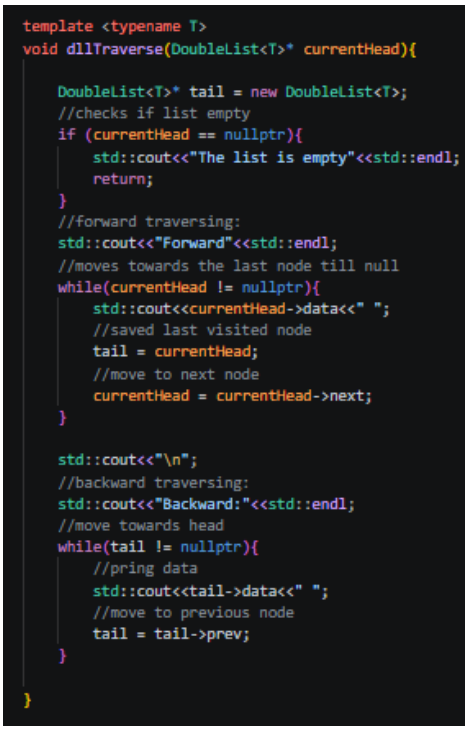
	Console	`deleting a node:`
		`GPEE101`
		Description: Searches for the character, then deletes it. Points the previous node to the next at the end.

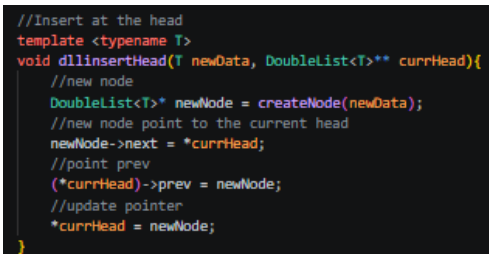
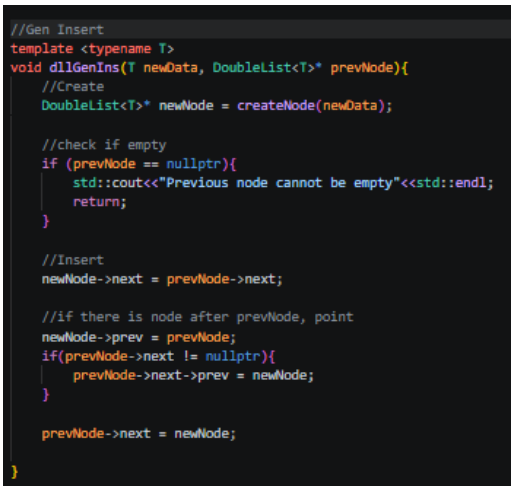
e.	Source Code	`std::cout<<"\ndeleting a node: \n"<<std::endl;` `sllDeleteNode('P', &head);`
	Console	`deleting a node` `GEE101`
f.	Source Code	`listTraversal(head);`
	Console	`GEE101` Description: Traverses and prints the remaining nodes.

Table 3-3. Code and Analysis for Singly Linked Lists

Screenshot(s)	Analysis


``
template <typename T>
class DoubleList{
    public:
        T data;
        DoubleList<T>* next = nullptr;
        DoubleList<T>* prev = nullptr;
};
 	
//create new node
template <typename T>
DoubleList<T>* createNode(T newData){
    DoubleList<T>* newNode = new DoubleList<T>;
 	
    newNode->data = newData;
    newNode->prev = nullptr;
    newNode->next = nullptr;
 	
    return newNode;
}
``
	Modified node creation class and function. Class now contains the “previous” pointer to point to a node before it. A constructor class alongside it to make node creation a tad more convenient.


``
template <typename T>
void dllTraverse(DoubleList<T>* currentHead){
 	
    DoubleList<T>* tail = new DoubleList<T>;
    //checks if list empty
    if (currentHead == nullptr){
        std::cout<<"The list is empty"<<std::endl;
        return;
    }
    //forward traversing:
    std::cout<<"Forward"<<std::endl;
    //moves towards the last node till null
    while(currentHead != nullptr){
        std::cout<<currentHead->data<<" ";
        //saved last visited node
        tail = currentHead;
        //move to next node
        currentHead = currentHead->next;
    }
 	
    std::cout<<"\n";
    //backward traversing:
    std::cout<<"Backward:"<<std::endl;
    //move towards head
    while(tail != nullptr){
        //pring data
        std::cout<<tail->data<<" ";
        //move to previous node
        tail = tail->prev;
    }
 	
}
``
	Traverse function now has an additional direction. When usually the traverse function can only go from head to tail, doubly linked lists allow for traversal from the tail back to the head.

	Does the same as singly list, the only difference is that the node points from both next and previous.
	Traverses till the end before adding a new node that points to null and the node previously before it.
	Works exactly like it has in singly list, but only after the new node was added, makes sure the correct pointers are created both next and previous depending on the position.

```
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head){

//if empty
if(*head == nullptr){
    return;
}
DoubleList<T>* currNode =*head;

//search
while(currNode != nullptr && currNode->data != findData){
    currNode = currNode->next;
}

//if data is not found
if(currNode == nullptr){
    return;
}

//if deleting head
if(currNode == *head){
    *head = currNode->next;
    if(*head != nullptr){
        (*head)->prev = nullptr;
    }
}
//points it back both next and prev
else {
    currNode->prev->next = currNode->next;
    if(currNode->next != nullptr){
        currNode->next->prev = currNode->prev;
    }
}

delete currNode;
}
```

Checks if empty, traverses and searches through the list until a match is made or the end of the list is reached. Saves the position, creates a temporary pointer which after the current node checker moves on, is deleted. It also checks for null pointers and checks if the head will be deleted.

Table 3-4. Modified Operations for Doubly Linked Lists

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Driver File:

VSCODES > CPE_Dimaguiba > SUPPLEMENTARY > Playlist > Playlist.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "playlist_commands.h"


int choice = 0;
int main(){
   DoubleList<std::string>* head = new DoubleList<std::string>;
   DoubleList<std::string>* second = new DoubleList<std::string>;
   DoubleList<std::string>* third = new DoubleList<std::string>;
   DoubleList<std::string>* currentSong = head; // so switching to next and previous wont change head

   //Create the circle loop
   head->data = "Let it Be";
   head->next = second;
   head->prev = third;

   second->data = "Welcome to The Black Parade";
   second->prev = head;
   second->next = third;

   third->data = "I Just Threw out the Love of My Dreams";
   third->prev = second;
   third->next = head;


   while (choice != 7){
      std::cout<<"   <Song Playlist>\nSelect an option:"<<std::endl;
      std::cout<<"1. Play All Songs\n2. Add a Song\n3. Delete a Song\n4. Play Loop(use at own risk (WILL LITERALLY LOOP))\n5. Play Next Track\n";
      std::cout<<"6. Play Previous Track\n7.Quit"<<std::endl;
      std::cout<<"Enter Choice here: ";

   while(!(std::cin>>choice)){
      std::cout<<"Invalid input!\n";
      std::cin.clear();
      std::cin.ignore(10000, '\n');
   }

   switch(choice){
      case 1:
         playAll(head);
         break;

      case 2:
      {
         std::string newTrack = "";
         std::cout<<"Enter Track Name to Add: ";
         std::cin.ignore();
         std::getline(std::cin, newTrack);
         addSong(std::string(newTrack), &head);
         break;
      }
       case 3:
       {
          std::string delTrack = "";
          std::cout<<"Enter Track Name to Delete: ";
          std::cin.ignore();
          std::getline(std::cin, delTrack);
          deleteSong(std::string(delTrack), &head);
          break;
       }
       case 4:
       playLoop(head);
       break;

       case 5:
       playNext(&currentSong);
       break;

       case 6:
       playPrev(&currentSong);
       break;

       case 7:
          break;

       default:
          std::cout<<"Invalid Choice\n";

       }

    }
    return 0;
}

/*
Create a playlist using multiple songs. done
Add songs to the playlist. done
Remove a song from the playlist. done
Play songs in a loop (for this activity, we will print all the songs once).*/

/*
Design the basic structure that supports circular data representation. done
After that, implement the insert and delete functions in the structure. done
Implement a function for traversing the playlist. done*/
```

Header File:

VSCODES > CPE_Dimaguiba > SUPPLEMENTARY > Playlist > C playlist_commands.h > ...

```
#ifndef PLAYLIST_COMMANDS_H
#define PLAYLIST_COMMANDS_H

#include<string>

template <typename T>
class DoubleList{
    public:
    T data;
    DoubleList<T>* next = nullptr;
    DoubleList<T>* prev = nullptr;
};


template <typename T>
void playLoop(DoubleList<T>* head){
    // if playlist is empty
    if (head == nullptr){
        std::cout<<"Playlist is empty!"<<std::endl;
        return;
    }

    while (true){
        std::cout<<"Now playing: "<<head->data<<std::endl<<std::endl;
        head = head->next;
    }

}

template <typename T>
void playAll(DoubleList<T>* head){
    // if playlist is empty
    if (head == nullptr){
        std::cout<<"Playlist is empty!"<<std::endl;
        return;
    }

    //stops the loop when all songs are played
    DoubleList<T>* firstSong = head;
    while (true){
        std::cout<<"Now playing: "<<head->data<<std::endl<<std::endl;
        head = head->next;
        if(head == firstSong){
            return;
        }
    }
}

template <typename T>
void playNext(DoubleList<T> ** currHead){
    // if playlist is empty
    if (*currHead == nullptr){
        std::cout<<"Playlist is empty!"<<std::endl;
        return;
    }

```

```
57      std::cout<<"Now playing: "<<(*currHead)->next->data<<std::endl<<std::endl;
58      (*currHead) = (*currHead)->next;
59  }
60
61  template <typename T>
62  void playPrev(DoubleList<T> ** currHead){
63      // if playlist is empty
64      if (*currHead == nullptr){
65          std::cout<<"Playlist is empty!"<<std::endl;
66          return;
67      }
68
69      std::cout<<"Now playing: "<<(*currHead)->prev->data<<std::endl<<std::endl;
70      *currHead = (*currHead)->prev;
71  }
72
73  template <typename T>
74  void addSong(T newData, DoubleList<T>** head){
75
76          //in an empty playlist
77      if (*head == nullptr){
78          DoubleList<T>* createSong = new DoubleList<T>;
79          createSong->data = newData;
80          createSong->next = createSong;
81          createSong->prev = createSong;
82          *head = createSong;
83          return;
84      }
85
86      DoubleList<T>* currSong =  *head;
87      DoubleList<T>* createSong = new DoubleList<T>;
88      createSong->data = newData;
89
90      //get to the end
91      while (currSong->next != *head){
92          currSong = currSong->next;
93      }
94
95    //connect
96    currSong->next = createSong;
97    createSong->next = *head;
98    createSong->prev = currSong;
99    (*head)->prev = createSong;
100 }
101
102 template <typename T>
103 void deleteSong(T findData, DoubleList<T>** currHead){
104     //if empty
105     if (*currHead == nullptr){
106         std::cout<<"Nothing to Delete!"<<std::endl;
107         return;
108     }
109
```

```
110     DoubleList<T>* current = *currHead;
111     DoubleList<T>* first = *currHead;
112     //search
113     while(current->data != findData){
114         if (current->next == first){
115             std::cout<<"Cannot find track"<<std::endl;
116             return;
117         }
118         current = current->next;
119     }
120 
121     //if theres only one node:
122     if (current->prev == *currHead && current->next == *currHead){
123         delete current;
124         *currHead = nullptr;
125         return;
126     }
127 
128     //if deleting head
129     if(current == *currHead){
130         *currHead = current->next;
131         (*currHead)->prev = current->prev;
132         (*currHead)->prev->next = *currHead;
133         }
134 
135     //points it back both next and prev
136     else {
137         current->prev->next =current->next;
138         current->next->prev = current->prev;
139         }
140 
141     delete current;
142 
143 }
144 #endif
```

Outputs:

Title:

```
    <Song Playlist>
Select an option:
1. Play All Songs
2. Add a Song
3. Delete a Song
4. Play Loop(use at own risk (WILL LITERALLY LOOP))
5. Play Next Track
6. Play Previous Track
7.Quit
Enter Choice here:
```

Play All:

```
Enter Choice here: 1
Now playing: Let it Be

Now playing: Welcome to The Black Parade

Now playing: I Just Threw out the Love of My Dreams
```

Add a Song:

```
Enter Choice here: 2
Enter Track Name to Add: All I Wanted
```

```
Now playing: I Just Threw out the Love of My Dreams

Now playing: All I Wanted
```

Play Next/Previous:

```
Enter Choice here: 5
Now playing: Welcome to The Black Parade
```

```
Enter Choice here: 6
Now playing: Let it Be
```

Loop:

```
Now playing: Welcome to The Black Parade

Now playing: I Just Threw out the Love of My Dreams

Now playing: All I Wanted

Now playing: Let it Be

Now playing: Welcome to The Black Parade

Now playing: I Just Threw out the Love of My Dreams

Now playing: All I Wanted

Now playing: Let it Be

Now playing: Welcome to The Black Parade
```

Summary of Lessons Learned:

Linked lists are a great alternative to arrays in terms of dynamically connecting or linking nodes, traversing, and reading data. It also allows for complete deletion of an element or object without leaving a shell or having to delete the entire list. Appending and inserting between a collection of nodes is easier compared to arrays who don't have that function. They also have properties similar or even just like pointers.

Analysis of the Procedure:

The program is built from a combination of 2 files. Although linked lists CAN be programmed into one file, the utility of having a template that can dynamically change with whatever data type it accepts and the cleaner and more organized approach compared to putting all your eggs in one basket. Debugging it would be simpler as well. Linked lists are a set of nodes that start from a head, reaching the tail. This means, a node must be pointing to another node, otherwise it must point to null. Singly Lists are an example of single entry and exit with only one direction to follow. Traversing can only go in one direction(as seen in the code), whenever a new head node is created, the address of the head is changed into the new one or points to the new one. This also applies when adding or deleting nodes.

The only difference with a doubly linked list is that it goes both ways. It points both to the next and previous nodes. This would mean 2 exit points. Traversing can be done in both directions, when adding new heads, nodes or deleting nodes, instead of pointing to the next node, one must also point to a previous node. Modifying the Singly List into a Doubly List isn't that much different. Only with the added new properties.

Analysis of the Supplementary Activity:

It follows the exact blueprint of a Doubly Linked List. Only this time, instead of having a null pointer at the ends, they circle around a loop. Traversing is the same, but without anywhere to stop, an extra step or checker was added when a loop is complete. The same methods apply when adding, deleting, or getting the next song. From getting the address and traversing, to creating and pointing front and back. One must be weary of the loop. One pointer going to null may break the system.

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?

Linked List is a great alternative to arrays that offers several features that arrays can't provide. However, this comes with its own challenges; being articulate and concise is the most important quality one must have especially when many pointers come into play. I would say I did decent, but not too great. There are some areas I have to wrap my head around, especially the deletion of nodes. But all in all, I could say I understand the concept.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Figuring out how to use strings for the header file.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **For some reason, even if you input a string into the parameters, the linked list doesn't really pick it up and turns it into a character unless you SPECIFICALLY declare it in parenthesis. After that, original ideas and code functions are done by me.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.